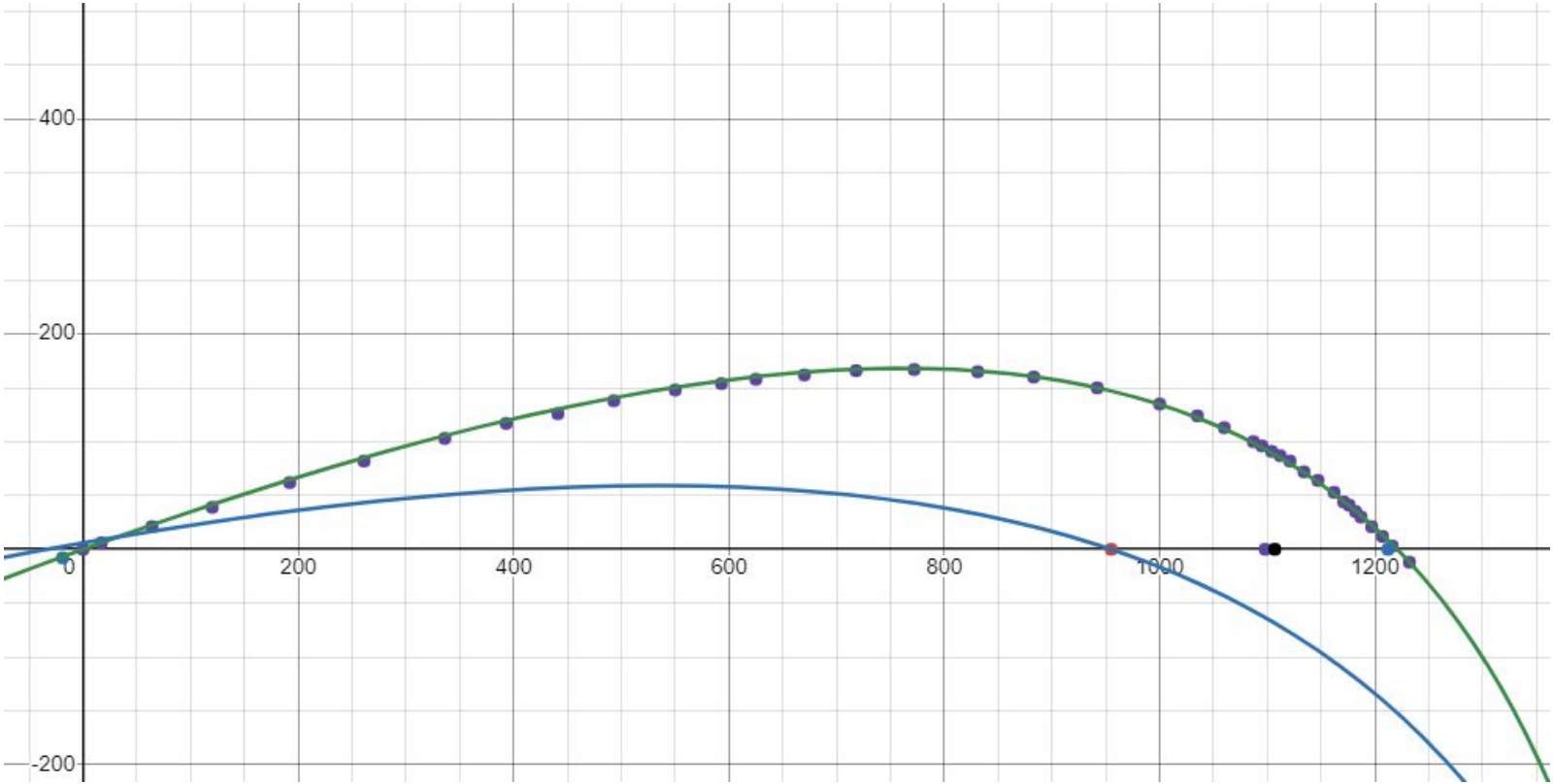
Ballistics Calculator



Prerequisite Knowledge Required:

AP Calculus

- Newton’s method
- Finding derivatives

AP Physics

- Deriving Formulas
- Drag

Computer Science

- PID

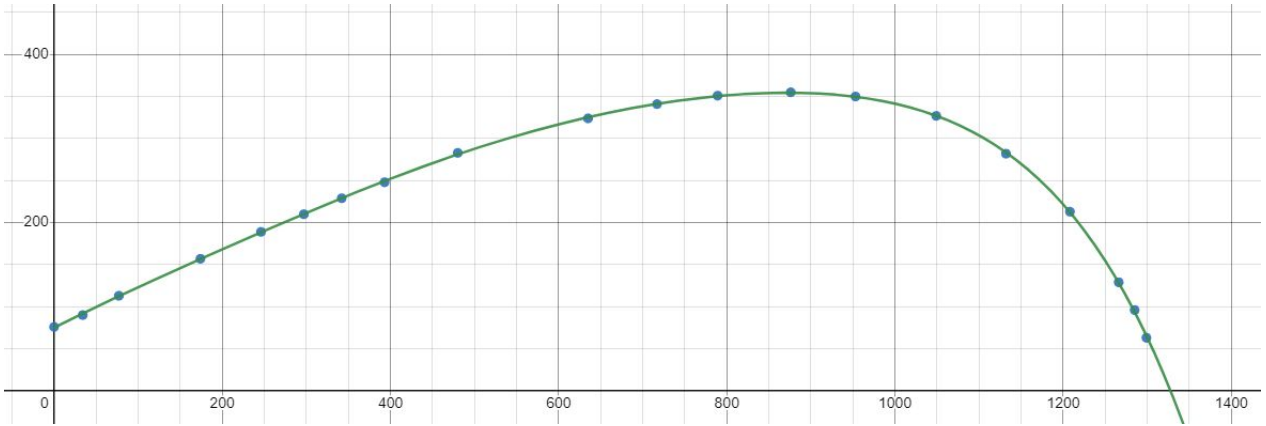
Additional

- Statistical Regression using Desmos
- Desmos knowledge
- Distance Formula / Pythagorean Theorem
- Trig Functions
- Atan2

The Physics

We know that the cannon shot experience linear drag based on the arc of the projectile, so standard projectile math doesn't apply here

This is what complicates it massively, as the drag force changes depending on the velocity of the projectile.



The Physics

So the first step is to derive an equation that relates the Y position of the projectile to the X position of the projectile:

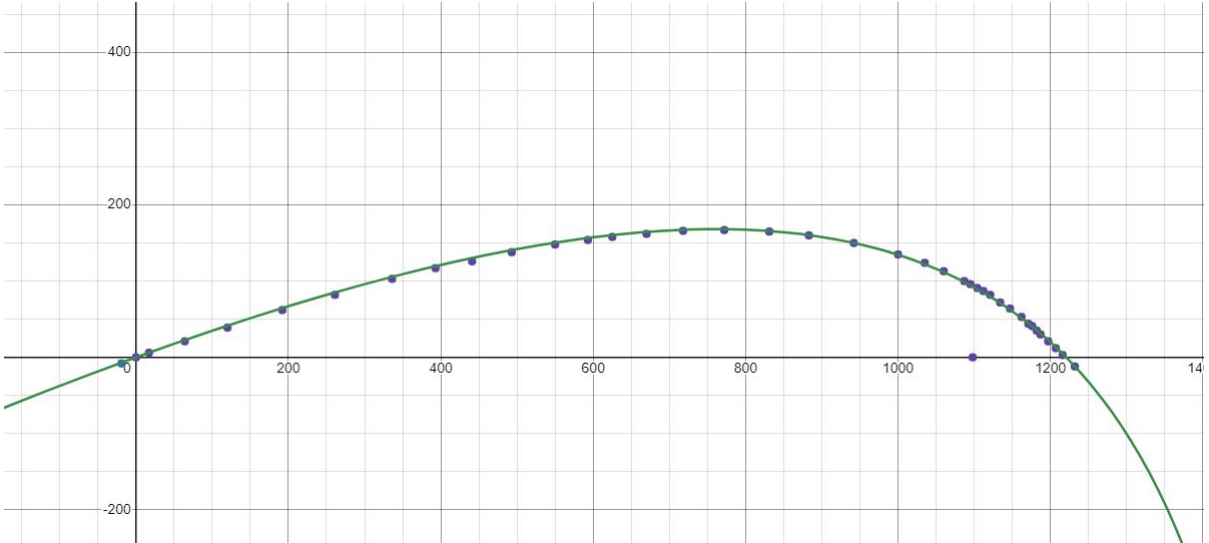
$$\frac{d_{rag2}\left(\left|S_{peed2}\right| \cdot \sin\left(\theta_{testAngle}\right)\right) + m_{ass2}g_{ravity2}}{d_{rag2}\left(\left|S_{peed2}\right| \cdot \cos\left(\theta_{testAngle}\right)\right)} x + \frac{m_{ass2}^2 g_{ravity2}}{d_{rag2}^2} \ln\left(\left|1 - \frac{d_{rag2}x}{m_{ass2}\left(\left|S_{peed2}\right| \cdot \cos\left(\theta_{testAngle}\right)\right)}\right|\right) + h$$

In this fase all of these equals the Y position of the Projectile based on the X position. This is nice and useful, but it is only useful if we know any of these physics constants.

Lucky for us, we know from the game's code that the projectile experiences 0.99/t drag, and 1/t gravity, so we can change those to seconds and get 0.0495 for drag, and 0.05 for gravity.

Statistical Regression

Even though we have values for the Gravity and drag, we are missing the velocity and mass, unfortunately the best way to get this value is by using statistical regression. In my case what I did was I plotted points along the curve of a cannon shot at a known angle and graphed that in desmos



Statistical Regression

$$y_2 \sim \frac{d_{rag2}\left(\left|S_{peed2}\right| \cdot \sin\left(\theta_{angle}\right)\right) + \left|m_{ass2}\right| g_{ravity2}}{d_{rag2}\left(\left|S_{peed2}\right| \cdot \cos\left(\theta_{angle}\right)\right)} x_2 + \frac{\left|m_{ass2}^2\right| g_{ravity2}}{d_{rag2}^2} \ln\left(1 - \frac{d_{rag2} x_2}{m_{ass2}\left(\left|S_{peed2}\right| \cdot \cos\left(\theta_{angle}\right)\right)}\right)$$

REGRESSION PARAMETERS

$S_{peed2} = 15.26032$ $m_{ass2} = 5.36303$

STATISTICS

$R^2 = 0.9989$

RESIDUALS

e_1

For me, this gave me values that I approximated to a mass of 5.5 and a velocity of 15.5 for a 20 barrel nethersteel cannon with 8 powder charges.

Finding Distance from Angles

This gives us an equation that allows us to find the Y pos based on X pos for our cannon fire, but this doesn't help us find the actual distance fired by the cannon at a set y position, or finding the Y pos based on the X pos.

We Can do this the same way you would find the zero of a function, but instead of 0 using ΔY or the difference in height you are shooting upon.

To find the zero of a function like this, unfortunately you cannot factor it or do any nice easy math, you have to use Newtons method to approximate the zero. This means we have to find the derivative of the function.

Newton's Method - Distance from Angle

$$g_{prime}(x) = \frac{d_{rag2}\left(\left|S_{peed2}\right| \cdot \sin\left(\theta_{testAngle}\right)\right) + m_{ass2}g_{ravity2}}{d_{rag2}\left(\left|S_{peed2}\right| \cdot \cos\left(\theta_{testAngle}\right)\right)} + \frac{m_{ass2}^2 g_{ravity2}}{d_{rag2}^2} \cdot \frac{-\frac{d_{rag2}}{m_{ass2}\left(\left|S_{peed2}\right| \cdot \cos\left(\theta_{testAngle}\right)\right)}}{1 - \frac{d_{rag2}x}{m_{ass2}\left(\left|S_{peed2}\right| \cdot \cos\left(\theta_{testAngle}\right)\right)}}$$

By setting the seed value to 1100 and repeating Newton's method about 10 times, you get an accurate estimate for the distance traveled based on the angle.

$a_0 = 1100$

-10

$$a_1 = a_0 - \frac{g(a_0)}{g_{prime}(a_0)}$$

$$a_2 = a_1 - \frac{g(a_1)}{g_{prime}(a_1)}$$

$$a_3 = a_2 - \frac{g(a_2)}{g_{prime}(a_2)}$$

$$a_4 = a_3 - \frac{g(a_3)}{g_{prime}(a_3)}$$

Finding Angle from Distance

Now going back to the original formula, we should set θ to be X as now the Distance is known

$$t_x = 955$$

$$h = -6$$

$$j(x) = \frac{d_{rag2}\left(\left|S_{peed2}\right| \cdot \sin(x)\right) + m_{ass2}g_{ravity2}}{d_{rag2}\left(\left|S_{peed2}\right| \cdot \cos(x)\right)} t_x + \frac{m_{ass2}^2 g_{ravity2}}{d_{rag2}^2} \ln\left(1 - \frac{d_{rag2}t_x}{m_{ass2}\left(\left|S_{peed2}\right| \cdot \cos(x)\right)}\right) - h$$

$$j_{prime}(x) = t_x\left(\sec^2(x - 0.022934)\right) + \frac{m_{ass2}g_{ravity2}t_x \sin(x - 0.022934)}{m_{ass2} \cdot S_{peed2}\cos^2(x - 0.022934)} + \frac{m_{ass2}^2 g_{ravity2}}{d_{rag2}^2} \frac{1}{\left(1 - \frac{d_{rag2} \cdot t_x}{m_{ass2} \cdot S_{peed2}\cos(x - 0.022934)}\right)} \cdot -\left(\frac{d_{rag2}t_x \sin(x - 0.022934)}{m_{ass2}S_{peed2}\cos^2(x - 0.022934)}\right)$$

Using this, we again find the derivative of the function, and apply Newton's method again to 10 iterations and we get the angle.

(in my case I had to apply a fudge factor to my derivative to get it working but if you do it right then you should be fine)

PID and the Cannon

Using CC:Tweaked you can write lua code that directly interacts with create objects, most importantly is the speed controller.

By setting the speed to `(kP * error) + (kD * prevError)` when `error = targetPos - currentPos` you can make the cannon move to a specified position fairly accurately. Just know that there is a limit to the precision of the cannon being only 2 decimal places so that is a major limiter.

I found that kP should be about 1.5 and kD should be about 1, with kI being unnecessary in this case.

PID and the Cannon

Then to track the position of the cannon, just take the time elapsed between the changes of speed and multiply that by the speed, account for the gear ratio and the minute to second conversion and you get the change in position of the cannon.

Make sure to also clamp the speed to [-256,256] and after the end of the control you set the speed controller to a speed of 0.

Create has a built in limit that prevents you from changing the speed at a rate faster than 0.21s or it will break the speed controller. So make sure have a sleep(0.21) inside your PID loop.

Accounting for Canon Length

The length of the cannon should not be ignored, with it changing the height and distance of the shot by a considerable amount.

In my case I use a Canon 28 blocks long, so After I calculate the angle once, I will add $28\sin(\theta)$ to ΔY and subtract $28\cos(\theta)$ to the distance.

Converting Coordinates to Pitch and Yaw

Using the distance formula on ΔZ and ΔX you get the distance between the cannon and the target position, but make sure you include ΔY when setting the pitch using the difference in the cannonY and targetY.

Yaw can be obtained by using Atan2(ΔX,ΔZ). With the PID personally I wouldn't worry about making it turn left or right depending on the target's target yaw and the cannon's current yaw as the time save is minute and the math is very annoying to formulate but if you want you can figure that out.

Additional Considerations

When making your cannons, think about setting some fail safes so the cannon doesn't fire at it’s own reload system

Also consider making it controlled by a pocket computer, sending it coords and it fires at them.

You can use a player detector to make it target players as well as coords.

Using a chatbox you can print debug information directly to you, or you can use it to control the canon by saying keyphrases

The cannon is limited to 2 decimal places so it may not be perfectly accurate, and the projectile takes up to 10s to land, so it isn't great at attacking moving targets like players